

FindHome - Real Estate Platform



SUBMITTED BY

Shahid Khan (2263083)
&
Attaullah Khan (2263051)

SUPERVISED BY

Mr. Tariq Ahmad

DEPARTMENT OF COMPUTER SCIENCE

Government GDC Kabal Swat, Affiliated with

UNIVERSITY OF SWAT

Session 2022-2026 (Batch 06)

Thesis Approval

This is to certify that we have studied this thesis entitled "FindHome - Real Estate Platform" submitted by “Shahid Khan and Attaullah Khan” in partial fulfilment of the requirements for the award of the degree of BS in Computer Science and has been approved by the supervisory committee.

Approval Committee

Tariq Ahmad (Project Supervisor)

Lecturer/Assistant Professor

Department of Computer Science

Government GDC Kabal Swat.

Signature

External Examiner

Signature

Ghais Ud Din (Head of Department)

Lecturer/Assistant Professor

Department of Computer Science

Government GDC Kabal Swat.

Signature

Declaration of Originality

We hereby declare that the work accomplished in this thesis is the result of our research carried out in the Department of BS Computer Science, Government GDC Kabal Swat. The thesis has not been published previously nor does it contain any material from published sources that can be considered as a violation of international copyright law. Moreover, we also declare that we are aware of the terms Copyright and Plagiarism, and if any copyright violation identified in this work, we shall be held responsible for the consequences of any such violation.

Shahid Khan

Attaullah Khan

Registration No. (2263083)

Registration No. (2263051)

Signature:

Signature:

Dedication

I would like to dedicate this project to the following people, without whom this effort would not have been possible.

Our parents, for their unwavering love, encouragement, and support throughout our entire academic journey. Your belief in us gave us the strength to persevere through challenges and achieve our goals.

Our supervisor, **Mr. Tariq Ahmad**, whose guidance and expertise have been instrumental in shaping the direction of this project. Thank you for sharing your knowledge and pushing us to explore new ideas.

Our friends and classmates, for being our pillars of strength during the highs and lows of this final year. Your camaraderie and laughter made the long hours of research and study more bearable.

The faculty members of the Computer Science Department, for their dedication to education and for fostering an environment of curiosity and critical thinking. We are grateful for the knowledge and skills we have acquired under your mentorship.

Thank you, everyone, for being a part of our lives and contributing to our academic and personal development. This project is as much yours as it is ours.

With heartfelt gratitude,

Shahid Khan & Attaullah Khan

Acknowledgements

In Almighty Allah's memory, who created the entire world. We are very thankful to Him who, in the form of wisdom, encouraged us and gave us strength and courage to complete this project and thesis.

We will never forget the advice from **Mr. Tariq Ahmad**, a passionate personality at the Department of Computer Science, Government GDC Kabal Swat, who led us to complete this project successfully. We always respect the endless patience displayed by our loving parents, brothers, sisters, relatives, and all the family members who supported us throughout the entire journey of education.

May this work of ours be successful and beneficial! Ameen!

Shahid Khan & Attaullah Khan

Abstract

“FindHome - Real Estate Platform” built using VS Code and React (Vite), Node.js/Express, and Socket.IO, powered by MongoDB managed through the Prisma ORM, the platform provides a simple yet powerful interface for real estate listing, discovery, and communication. A FindHome website acts as an online platform created to streamline and simplify the process of buying, renting, and listing properties for individuals and property owners. It acts as a central point where buyers, renters, and property owners can view and manage information about property listings, real-time messaging, saved posts, and administrative oversight.

This platform guarantees that data is always available and accurate, making it easier to monitor property availability and listing status. It improves communication by providing features like real-time chat, typing indicators, and instant messaging for interactions among buyers, renters, and property owners. Moreover, the system offers secure access through role-based permissions, protecting data privacy and adhering to modern web security standards.

The platform streamlines the handling of property records, lessens the administrative burden on owners, and gives buyers and renters access to current and comprehensive property information. Modern web technologies make it scalable, easy to use, and compatible across desktop and mobile browsers.

In summary, FindHome is a valuable resource that boosts the operational effectiveness of the real estate process by digitizing and automating essential property listing and discovery tasks.

Table of Contents

Thesis Approval	i
Declaration of Originality	ii
Dedication	iii
Acknowledgements	iv
Abstract.....	v
Table of Contents	vi
List of Figures.....	xiv
List of Tables	xv
Chapter 1 Introduction.....	1
1.1 FindHome - Real Estate Platform	1
1.2 Objective	1
1.4 Problem Statement	1
1.4.1 Fragmented Listings.....	2
1.4.2 Lack of Direct Communication	2
1.4.3 Inadequate Filtering and Detail.....	2
1.4.4 Lack of Integration.....	2
1.4.5 Inadequate Reporting	2
1.4.6 Security Concerns:	3

1.4.7	Poor Owner Management Tools	3
1.5	Scope	3
1.6	Significance	3
1.6.1	Centralized Data Management:.....	3
1.6.2	Real-Time Engagement	4
1.6.3	Enhanced Communication:	4
1.6.4	Scalability and Flexibility:	4
1.6.5	Better User Engagement:	4
1.7	Key Features of FindHome	5
Chapter 2	Requirement Analysis.....	6
2.1	Introduction	6
2.2	Requirements Analysis:.....	6
2.2.1	Requirement Gathering Methods:	6
2.3	Stakeholder Identification	7
2.4	Functional Requirements.....	7
2.4.1	User Authentication and Account Management:	8
2.4.2	Property Listing Management:	8
2.4.3	Communication Module:	8
2.4.4	Reporting and Analytics:	8
2.4.5	User Access Control:	8

2.5	Non-Functional Requirements	9
2.5.1	Performance:	9
2.5.2	Scalability:	9
2.5.3	Usability:.....	9
2.5.4	Reliability:.....	9
2.5.5	Compatibility:	10
2.6	System Architecture	10
2.6.1	Architecture Components	10
2.6.1.1	User Interface:.....	10
2.6.1.2	Database:.....	10
Chapter 3	System Designing	11
3.1	Introduction to Designing.....	11
3.2	Data Analysis and Modelling.....	11
3.3	Database Design.....	11
3.3.1	Entities or Data Element:	11
3.3.2	Relationship:	12
3.3.3	Entities:	12
3.3.3.1	User Entity:	12
3.3.3.2	Post Entity:	13
3.3.3.3	PostDetail Entity:	13
3.3.3.4	SavePost Entity:	14
3.3.3.5	Chat Entity:	14
3.3.3.6	Message Entity:.....	15
3.3.3.7	ContactMessage Entity	15

3.3.3.8	Entity Relationship Diagram (ERD) for Student Management System (RMS):.....	16
3.4	Standardization.....	17
3.5	UML Use Case Diagram.....	18
3.5.1	Actors and Use Cases.....	18
3.6	Data Dictionary	19
3.6.1	User Table.....	19
3.6.2	Post Table.....	20
3.6.3	PostDetail Table.....	21
3.6.4	SavePost Table.....	21
3.6.5	Chat Table.....	22
3.6.6	Message Table	22
3.6.7	ContactMessage Table	22
Chapter 4	Testing.....	24
4.1	Testing.....	24
4.2	Overview of Testing.....	24
4.3	Testing Strategy.....	25
4.3.1	Testing of Individual Components (Unit Testing).....	26
4.3.2	Test of Components Working Together (Integration Testing)	27
4.3.3	Test of the Entire System (System Testing)	28
4.3.4	Test of System Functionality (Validation Testing).....	29
4.3.5	Security Testing	29

4.3.6	Performance Testing	30
Chapter 5	Tools and Technology	32
5.1	Introduction to Tools and Technology	32
5.2	Frontend Technologies	33
5.2.1	React 19	33
5.2.2	Vite 7.....	33
5.2.3	Tailwind CSS 4	34
5.2.4	React Router DOM 7	34
5.2.5	Axios	35
5.2.6	Leaflet.js.....	35
5.2.7	Socket.IO Client.....	35
5.3	Backend Technologies	36
5.3.1	Node.js	36
5.3.2	Express.js	37
5.3.3	Prisma ORM 6	37
5.3.4	JSON Web Tokens (JWT)	38
5.3.5	bcrypt	38
5.3.6	Resend.....	38
5.3.7	Socket.IO Server	39
5.4	Database	39

5.5	Development Tools	40
5.5.1	Visual Studio Code (VS Code)	40
5.5.2	Postman.....	41
5.5.3	Git and GitHub.....	41
5.5.4	MongoDB Atlas	42
Chapter 6	User Interface Design	43
6.1	Introduction to User Interface Design	43
6.2	Public Pages	43
6.2.1	Home Page	44
6.2.2	About Page.....	44
6.2.3	List Page.....	44
6.2.4	Search Results Page	44
6.2.5	Property Detail Page	45
6.2.6	Login Page	45
6.2.7	Register Page	45
6.2.8	Forgot Password Page.....	45
6.2.9	Contact Page	46
6.2.10	Privacy Policy Page	46
6.2.11	Terms of Service Page	46
6.2.12	Sitemap Page.....	46

6.3	Dashboard Pages	47
6.3.1	Overview Page (<i>Both Users and Admins</i>)	47
6.3.2	My Properties Page (<i>Owner only or Admin</i>).....	47
6.3.3	Add Property Page (Both Users and Admins)	48
6.3.4	Edit Property Page (<i>Owner only or Admin</i>)	48
6.3.5	Saved Posts Page (<i>Both Users and Admins</i>)	48
6.3.6	Messages Page (Both Users and Admins)	49
6.3.7	Profile Page (Both Users and Admins).....	49
6.3.8	Users Page (Admin only).....	50
6.3.9	Property Detail Page - Dashboard View (Both Users and Admins).....	50
6.3.10	Queries Page (<i>Admin only</i>)	50
Chapter 7	Conclusion and Future Work	52
7.1	Conclusion of a Student Management System (SMS) Implementation.....	52
7.2	Future Work	53
7.2.1	Integration with Payment Gateways	53
7.2.2	Development of a Mobile Application	53
7.2.3	Artificial Intelligence-Driven Property Recommendations	53
7.2.4	Customizable Dashboards.....	53
7.2.5	Cloud-Based Infrastructure and Deployment	54
7.2.6	Support for Multiple Languages	54

7.2.7	Advanced Search and Filtering	54
7.2.8	Extended Platform Scope	55
Chapter 8	References	56

List of Figures

Figure 3.3.1 User Entity.....	12
Figure 3.3.2 Post Entity	13
Figure 3.3.3 PostDetail Entity.....	13
Figure 3.3.4 SavePost Entity.....	14
Figure 3.3.5 Chat Entity.....	14
Figure 3.3.6 Message Entity	15
Figure 3.3.7 ContactMessage Entity.....	15
Figure 3.3.8 Entity Relationship Diagram (ERD) for Student Management System (RMS)	Error! Bookmark not defined.
Figure 3.4.1 Standardization	Error! Bookmark not defined.

List of Tables

Table 3.6.1 User Table.....	19
Table 3.6.2 Post Table	20
Table 3.6.3 PostDetail Table.....	21
Table 3.6.4 SavePost Table.....	21
Table 3.6.5 Chat Table.....	22
Table 3.6.6 Message Table	22
Table 3.6.7 ContactMessage Table.....	Error! Bookmark not defined. 2

Chapter 1 Introduction

1.1 FindHome - Real Estate Platform

A modern real estate platform called FindHome was created to improve and expedite the property search and listing process for buyers, renters, and property owners. In order to satisfy these needs, FindHome provides a complete solution by giving administrators, property owners, and buyers simple-to-use tools for effectively managing several facets of real estate transactions. The systematic organization, preservation, and administration of property-related data are facilitated and automated through this framework, thereby improving the accessibility and application of such information for buyers, sellers, and platform administrators.

1.2 Objective

The fundamental objectives of FindHome are to enhance the effectiveness, efficiency, and accuracy of the management of property listings and administrative processes within a modern real estate environment.

1.4 Problem Statement

The present real estate search process is fragmented and ineffective, resulting in numerous challenges associated with the management of property data, listings, communication among buyers, property owners, and administrative personnel. The system

predominantly operates in a manual or semi-automated manner, which gives rise to the following problems:

1.4.1 Fragmented Listings

The manual posting of property information across multiple platforms frequently leads to inaccuracies and inconsistencies. The process of updating records is labour-intensive and susceptible to human error.

1.4.2 Lack of Direct Communication

The existing platforms may lack embedded real-time communication features, thereby complicating the ability of buyers and property owners to interact directly. This deficiency delays effective negotiation and the provision of timely updates.

1.4.3 Inadequate Filtering and Detail

The management of property records, encompassing pricing, location, amenities, and status, is difficult. The retrieval and updating of listings occur at a sluggish pace, resulting in delays within the property discovery process.

1.4.4 Lack of Integration

The current platforms may not be synchronized with other vital tools such as interactive maps, proximity data, and real-time messaging, thereby producing a fragmented experience for users.

1.4.5 Inadequate Reporting

The generation of reports concerning property availability, listing status, and platform-wide statistics is problematic. The existing platforms may not offer customizable or real-time reporting functionalities, thereby complicating the ability of administrators to make well-informed decisions.

1.4.6 Security Concerns:

With the escalating volume of sensitive user and property information, current platforms may be deficient in robust security protocols, rendering them vulnerable to data breaches and unauthorized access.

1.4.7 Poor Owner Management Tools

As the quantity of listings and users expands, current platforms may encounter difficulties in accommodating the increased demand, resulting in diminished performance and potential system failures.

1.5 Scope

FindHome is a web-based platform created to streamline property listing and discovery for buyers, renters, and property owners. It is designed to handle property information effectively across multiple categories including residential, commercial, and land listings, serving users across desktop and mobile browsers.

1.6 Significance

The importance of FindHome is rooted in its capacity to simplify and improve the administration of real estate transactions. Here is why FindHome is essential:

1.6.1 Centralized Data Management:

Effectiveness: FindHome brings together all property-related information, including listing details, images, pricing, location coordinates, and status, into one straightforward platform. This consolidation minimizes the necessity for several fragmented listing sources and eliminates the need for manual record keeping, saving both time and effort.

Improved Decision Making: By organizing data in one place, FindHome decreases discrepancies and mistakes, guaranteeing that property information is precise and current.

1.6.2 Real-Time Engagement

Instant Communication: Buyers and property owners have the ability to communicate in real time through the embedded Socket.IO chat system, pinpointing opportunities and negotiating directly without leaving the platform.

Improved Decision Making: The platform offers proximity data including distances to schools, bus stops, and restaurants that enables buyers to make informed decisions tailored to their specific needs.

1.6.3 Enhanced Communication:

Efficient Interaction: Using FindHome makes it easier for buyers, renters, and property owners to communicate effectively. Messages, typing indicators, and real-time notifications can be shared quickly and reliably.

1.6.4 Scalability and Flexibility:

Expansion Control: As the number of listings and users increases, FindHome can effortlessly expand to handle more data and users without affecting its efficiency, owing to its three-tier independently deployable architecture.

Flexibility: The system is versatile and can be tailored to suit the specific requirements of various real estate markets, ranging from small communities to large regional marketplaces.

1.6.5 Better User Engagement:

Ease of Use: Users have the ability to retrieve property listings, saved posts, messages, and dashboard statistics via the internet, simplifying their organization and involvement in the property search process.

Feedback Systems: The platform incorporates a contact form functionality that enables users to submit enquiries and receive responses from administrators, aiding in the ongoing enhancement of the platform experience.

1.7 Key Features of FindHome

- **Administrators:** Administrators are in charge of the entire FindHome platform. They handle the oversight and maintenance of the system, manage user accounts, adjust access levels, and guarantee the accuracy and safety of data including listing verification and contact query management.

- **Property Owners:** Property owners use the system regularly to create and manage their listings, track listing status, and communicate with interested buyers and renters. They depend on the system to monitor listing activity and make informed decisions regarding pricing and availability.

- **User Profile:** Every registered user has a unique profile that includes personal details, contact information, property listings, saved posts, and chat history.

Chapter 2 Requirement Analysis

2.1 Introduction

This chapter offers an in-depth examination and planning of the requirements for the FindHome real estate platform, covering both its operational and technical features. It emphasizes the different responsibilities and roles of users, including administrators, property owners, and buyers/renters.

2.2 Requirements Analysis:

The process of analyzing requirements in software development is essential because it includes pinpointing and recording the desires and anticipations of those involved. This step lays the foundation for a successful project by ensuring that the end product meets its intended function and goal. Understanding the objectives of the platform, creating clear requirements, and transforming them into practical guidelines that guide the development process are all important steps. The following will discuss methods for requirements gathering.

2.2.1 Requirement Gathering Methods:

To make sure FindHome satisfies the requirements of everyone involved, different methods for gathering needs were used:

Interviews: Conversations were held with important stakeholders, like property owners, buyers, renters, and platform administrators, to learn about their unique needs and what they expect from the system.

Surveys: Surveys were sent out to a wider group, especially property seekers and owners, to gather information on their preferences, difficulties, and issues with existing real estate platforms.

Seminars: Seminars were held with potential users and developers to discuss how things are currently done, what problems they face, and what features they want in the new system.

Review of Documents: An examination of existing real estate platforms and their workflows was conducted to make sure all necessary features and requirements were included in the system design.

2.3 Stakeholder Identification

Identifying the key stakeholders is essential for understanding who will interact with the system and what their specific needs are. The primary stakeholders identified for FindHome include:

- **Administrators:** Responsible for managing user accounts, overseeing all property listings, handling contact form submissions, activating or deactivating user accounts, and ensuring compliance with platform standards.
- **Property Owners:** Need tools for creating and managing listings, tracking listing status, updating property details, and communicating with interested buyers and renters.
- **Buyers/Renters:** Require access to property listings, advanced search and filtering tools, saved posts, real-time messaging with property owners, and a secure authentication system.

2.4 Functional Requirements

Functional requirements define the particular actions and operations that the FindHome platform is required to facilitate. These include:

2.4.1 User Authentication and Account Management:

Register, update, and remove user profiles. Keep track of personal details, contact information, account status, and role-based access. Support a complete forgot-password flow including OTP generation, email delivery via Resend, OTP verification, and secure password reset.

2.4.2 Property Listing Management:

Allow authenticated users to create, update, and delete property listings. Manage property details including title, price, address, city, GPS coordinates, listing type (Buy/Rent), property type (Apartment/Home/Commercial/Land), images, and status (available, pending, sold, rented).

2.4.3 Communication Module:

Establish a real-time communication channel within the platform for interactions between buyers and property owners. Facilitate secure, instant dialogue between users through a Socket.IO powered chat system with typing indicators and unread message tracking.

2.4.4 Reporting and Analytics:

Generate reports focusing on property listing statistics, user activity, and platform-wide post status breakdowns. Offer dashboard tools for administrators to monitor platform performance and for property owners to track the status of their own listings.

2.4.5 User Access Control:

Enforce access control based on roles to guarantee that users can only view and perform actions pertinent to their positions. Enable administrators to manage user roles and account statuses. Offer a secure login mechanism using JWT tokens stored as HTTP-only cookies with role-based middleware protecting all sensitive API routes.

2.5 Non-Functional Requirements

The characteristics and elements of the system that developers must incorporate are referred to as non-functional requirements. Attributes such as reliability, ease of use, efficiency, ease of maintenance, and security are instances of non-functional requirements. They include:

2.5.1 Performance:

The system should enable concurrent use by numerous users without a decline in performance. The API shall respond to listing search queries within two seconds under normal load, and the Socket.IO server shall deliver messages with less than 500 ms latency on a local network.

2.5.2 Scalability:

The framework needs to be adaptable to manage rising numbers of listings, users, and data without necessitating a total architectural overhaul. The three-tier architecture of FindHome supports independent scaling of the React client, Express API, and MongoDB database.

2.5.3 Usability:

The interface shall be responsive across desktop, tablet, and mobile viewports. All interactive elements shall provide visual feedback including loading states, error messages, and success notifications to ensure a smooth user experience.

2.5.4 Reliability:

All API controllers implement try/catch error handling and return appropriate HTTP status codes to prevent any interruptions in accessing property data and administrative functions. The database connection is managed through a singleton Prisma client instance to ensure consistent and stable data access.

2.5.5 Compatibility:

The platform must work well with different devices, such as desktop computers, laptops, tablets, and mobile phones. It is ensured to work across different web browsers including Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge.

2.6 System Architecture

The structure of the system is detailed in this part, providing an overall summary of the FindHome real estate platform, including the different components and their interconnections.

2.6.1 Architecture Components

Here is a breakdown of the structural elements involved in the FindHome platform:

2.6.1.1 User Interface:

The front-end of FindHome is built using React 19 and Vite 7, resulting in a seamless and engaging experience for the user. Tailwind CSS 4 is used for utility-first styling, ensuring the platform is responsive and visually consistent across different devices and screen sizes. React Router DOM 7 handles client-side routing, while Axios manages all HTTP communication with the backend API. Leaflet.js renders interactive property location maps on detail pages.

2.6.1.2 Database:

A flexible document-oriented NoSQL database called MongoDB is used to store and manage information for users, property listings, chats, messages, and contact submissions, ensuring the accuracy and consistency of data throughout the platform. Prisma ORM 6 serves as the type-safe query layer between the Express API and MongoDB.

Chapter 3 System Designing

3.1 Introduction to Designing

The process of outlining a system's structure, including its modules, frameworks, components, and additional specifics, according to set criteria, is referred to as system design. This involves the steps of identifying, creating, and planning a system that meets the specific needs and criteria of the FindHome real estate platform. This chapter covers the database schema, entity-relationship model, API route design, and the UML use-case diagram that represents interactions between actors and system functions.

3.2 Data Analysis and Modelling

This is the starting role of the analysis model. The next step involves fundamental data modelling and analysis:

Entities or data elements.

Attributes of data objects.

Relationships between entities.

A data model diagram, known as an ERD, is produced by utilizing these frameworks.

3.3 Database Design

3.3.1 Entities or Data Element:

Any object possessing certain characteristics (properties) is referred to as an entity. This entity includes the data object along with all its attributes. FindHome uses MongoDB as its

database with Prisma ORM as the type-safe query layer. The schema defines seven models, each mapped to a MongoDB collection, using ObjectId for primary keys.

3.3.2 Relationship:

In a database, a relationship is a real link that joins various data elements (entities). This link outlines how information from one collection is associated with data from another. In FindHome, relationships are managed through ObjectId references defined in the Prisma schema, connecting users to posts, posts to details, chats to messages, and users to saved posts.

3.3.3 Entities:

We classify into the following types of entities:

- User
- Post
- PostDetail
- SavePost
- Chat
- Message
- ContactMessage

3.3.3.1 User Entity:

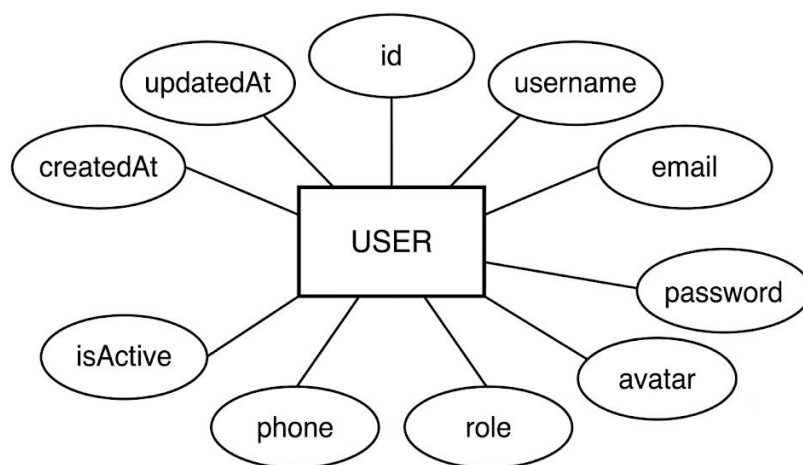


Figure 3.3.1 User Entity

3.3.3.2 Post Entity:

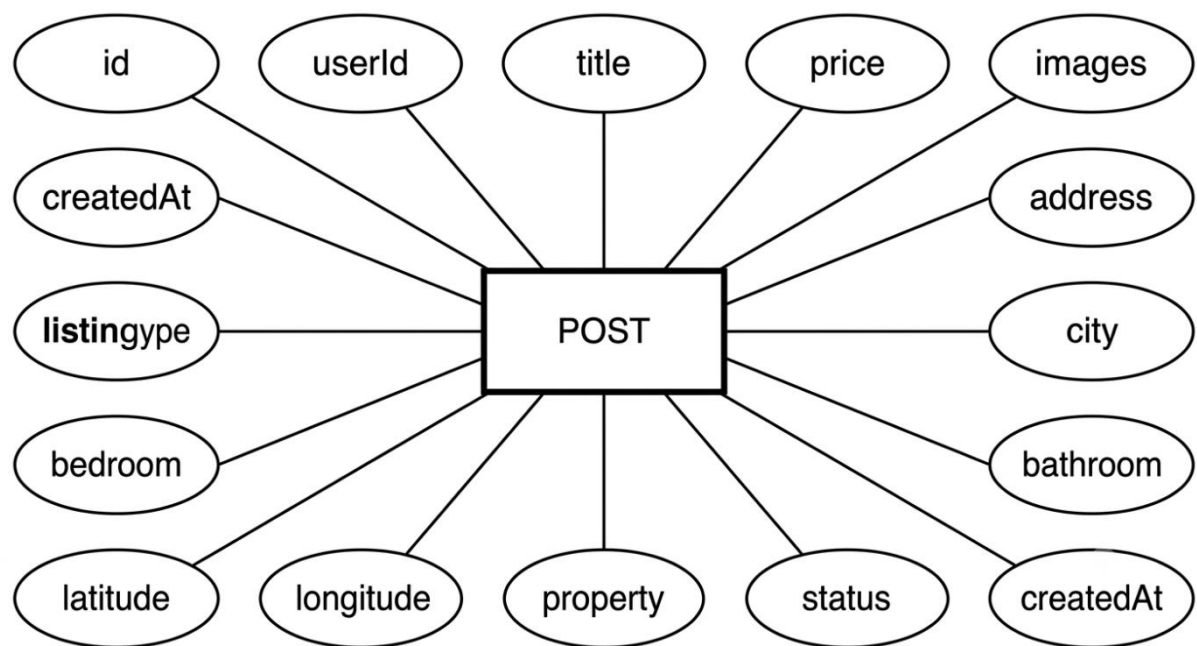


Figure 3.3.2 Post Entity

3.3.3.3 PostDetail Entity:

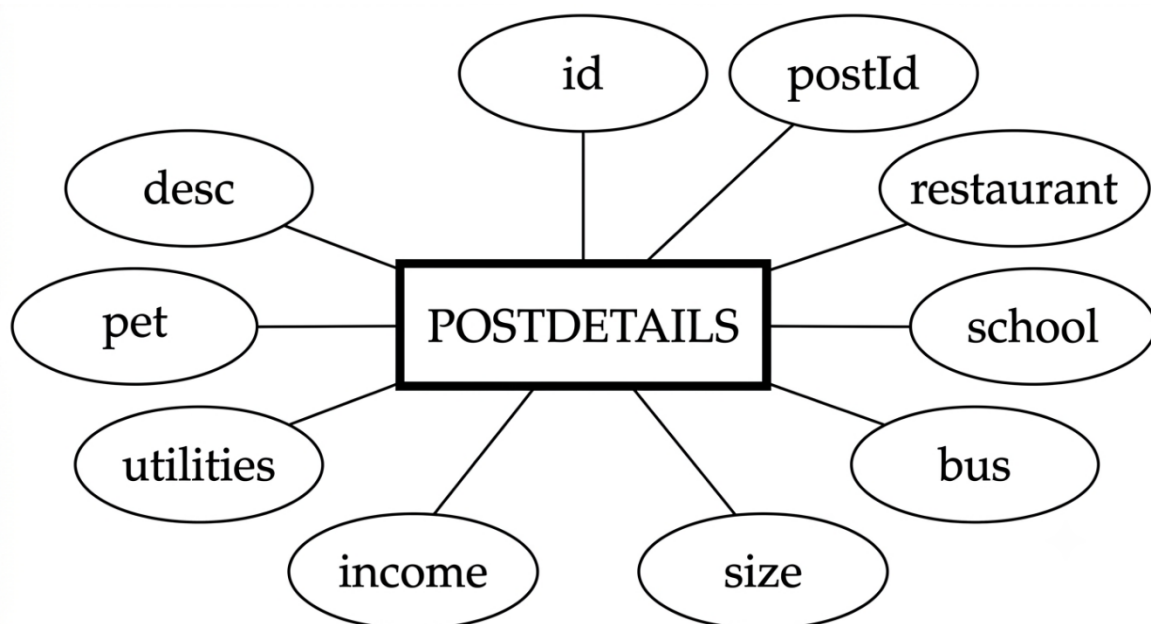
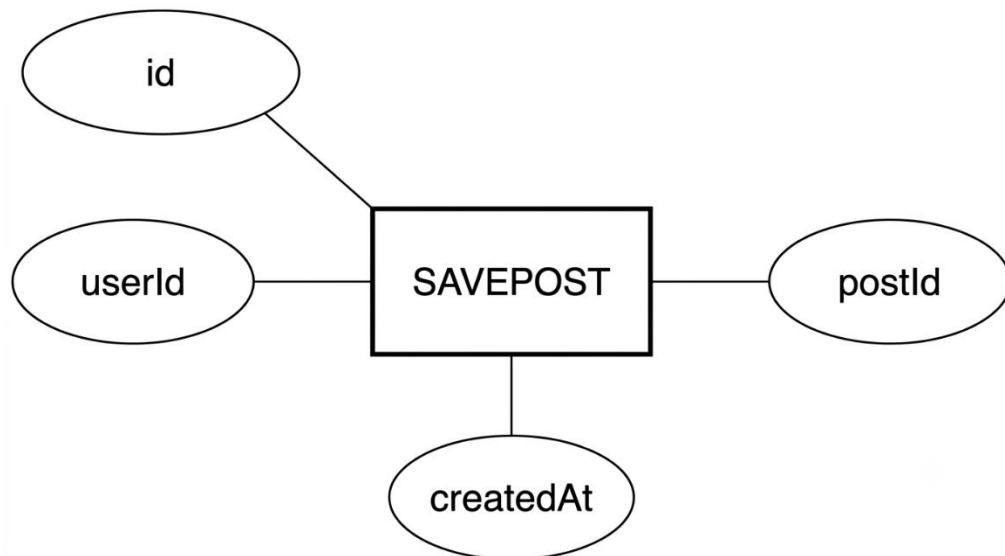
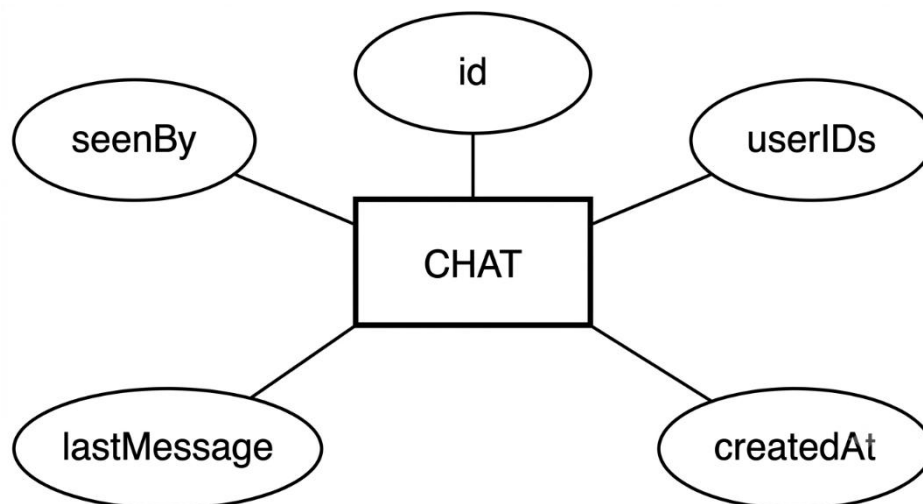


Figure 3.3.3 PostDetail Entity

3.3.3.4 SavePost Entity:*Figure 3.3.4 SavePost Entity***3.3.3.5 Chat Entity:***Figure 3.3.5 Chat Entity*

3.3.3.6 Message Entity:

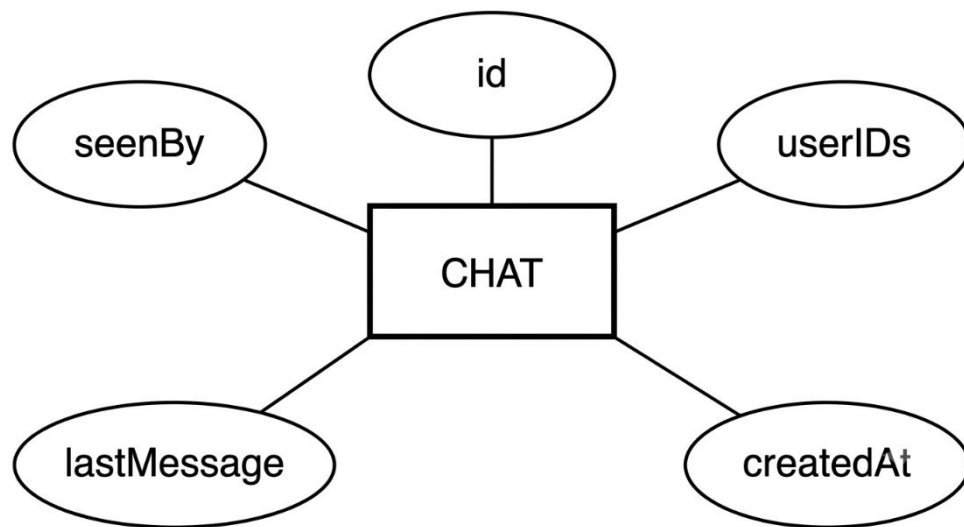


Figure 3.3.6 Message Entity

3.3.3.7 ContactMessage Entity

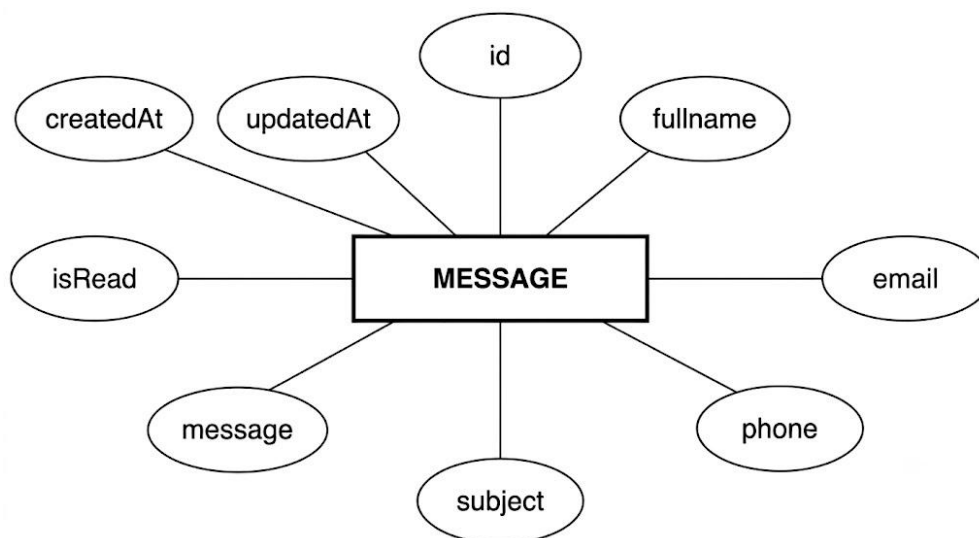


Figure 3.3.7 ContactMessage Entity

3.3.3.8 Entity Relationship Diagram (ERD) for Student Management System (RMS):

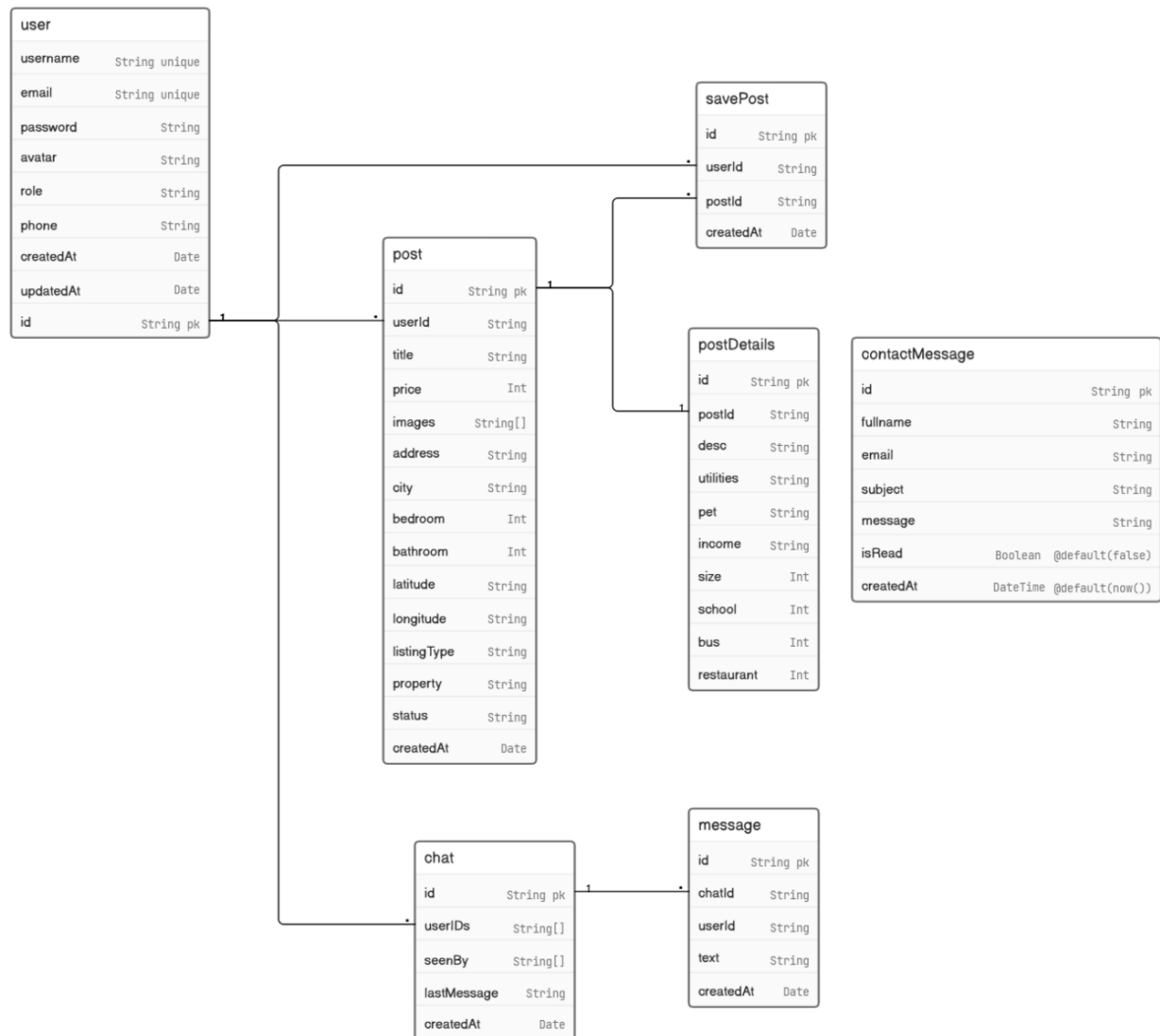


Figure 3.3.8 ER-Diagram Student Management System

3.4 Standardization

Standardization involves organizing information in a database to minimize duplication and enhance the accuracy of the data. In FindHome, the Prisma schema enforces standardization by separating core listing data (post model) from extended property details (postDetail model) through a one-to-one relationship. Similarly, chat participants and message history are stored in separate collections, reducing redundancy and improving query efficiency. By establishing connections between related collections through ObjectId references, the database remains compact and easier to maintain.

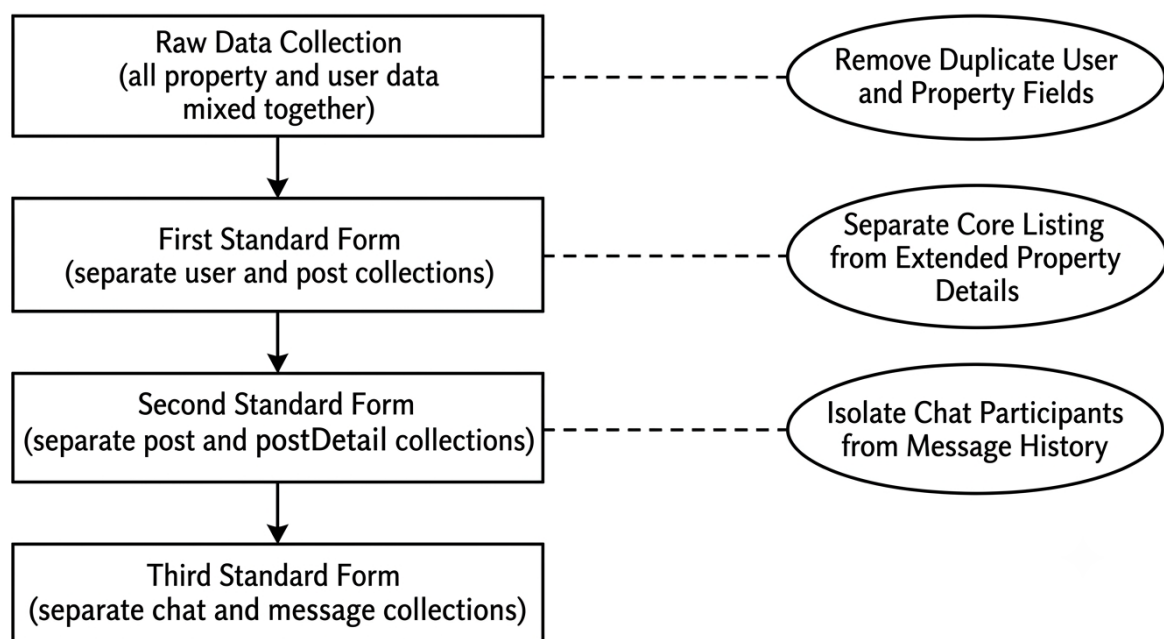


Figure 3.4.1 Standardization

3.5 UML Use Case Diagram

A use-case diagram showcases the connection between the user (actor) and the system. It outlines the operational mechanisms of the system and the level of interaction from the users towards it. It offers a broad perspective on the system's functional needs and depicts the various methods users have to engage with the system and the services it offers.

3.5.1 Actors and Use Cases

Anonymous Visitor:

- Browse property listings
- Search and filter listings by city, type, price, and bedrooms
- View property detail page with map and proximity data
- Submit contact form
- Register for an account
- Log in to the platform

Authenticated User (extends Anonymous Visitor):

- Save and unsave property listings
- Add a new property listing
- Edit own listing details and status
- Delete own listing
- Initiate and participate in real-time chat
- View own dashboard with statistics
- Manage profile information
- Apply forgot-password flow via OTP email

Administrator (extends Authenticated User):

- View and manage all registered users
- Activate or deactivate user accounts
- View and manage any property listing
- Manage and respond to contact form submissions
- View platform-wide listing statistics

3.6 Data Dictionary

A data dictionary serves as a central documentation resource that holds a detailed overview of the data elements utilized in a database or information system. It acts as a catalogue that offers details on every data field, such as its name, data type, description, constraints, and any other pertinent information. This allows developers and database managers to gain a comprehensive understanding of the database's data elements and their functional roles within the system, aiding in the maintenance of consistent, precise, and effective data handling across the platform.

3.6.1 User Table

Table 3.6.1 User Table

Field Name	Type of Data	Description
id	String (ObjectId)	Primary Key of User collection
username	String	Unique username of the registered user
email	String	Unique email address of the user
Email	Enum(USER/ADMIN)	Email address Define.
role	String	Defines the access role of the user
password	String	bcrypt-hashed password
avatar	String?	Optional profile image URL
phone	String?	Optional contact number
isActive	Boolean	Defines whether the account is active
createdAt	DateTime	Date and time of account creation
updatedAt	DateTime	Date and time of last update

3.6.2 Post Table

Table 3.6.2 Post Table

Field Name	Type of Data	Description
id	String (ObjectId)	Primary Key of Post collection
userId	String (ObjectId)	Foreign key referencing the User collection
title	String	Title of the property listing
price	Int	Price of the property
images	String[]	Array of image URLs for the listing
address	String	Street address of the property
city	String	City of the property
bedroom	Int?	Number of bedrooms
bathroom	Int?	Number of bathrooms
latitude	String	GPS latitude coordinate
longitude	String	GPS longitude coordinate
property	String	Property type: Apartments, Home, Commercial, Land
listingType	String	Listing type: Rent or Buy
status	String	Listing status: available, pending, sold, rented
createdAt	DateTime	Date and time of listing creation

3.6.3 PostDetail Table

Table 3.6.3 PostDetail Table

Field Name	Type of Data	Description
id	String (ObjectId)	Primary Key of PostDetail collection
postId	String (ObjectId)	Unique foreign key referencing the Post
desc	String?	Full description of the property
utilities	String?	Utilities policy of the property
pet	String?	Pet policy of the property
income	String?	Minimum income requirement
size	Int	Property size in square feet or metres
school	Int	Distance to nearest school in metres
bus	Int	Distance to nearest bus stop in metres
restaurant	Int	Distance to nearest restaurant in metres

3.6.4 SavePost Table

Table 3.6.4 SavePost Table

Field Name	Type of Data	Description
id	String (ObjectId)	Primary Key of SavePost collection
userId	String (ObjectId)	Foreign key referencing the User
postId	String (ObjectId)	Foreign key referencing the Post
createdAt	DateTime	Date and time when the post was saved

3.6.5 Chat Table

Table 3.6.5 Chat Table

Field Name	Type of Data	Description
id	String[] (ObjectId)	Primary Key of Chat collection
userIDs	String[] (ObjectId)	Array of participant user IDs
seenBy	String[] (ObjectId)	Array of user IDs who have seen the last
lastMessage	String?	Preview of the most recent message
createdAt	DateTime	Date and time of chat creation

3.6.6 Message Table

3.6.7 ContactMessage Table

Table 3.6.7 ContactMessage Table

Field Name	Type of Data	Description
id	String (cuid)	Primary Key of ContactMessage
fullName	String	Full name of the form submitter
email	String	Email address of the submitter
phone	String?	Optional phone number of the submitter
subject	String	Subject of the enquiry
message	String	Full message body of the enquiry
isRead	Boolean	Whether the submission has been read
createdAt	DateTime	Date and time of submission

updatedAt	DateTime	Date and time of last update
-----------	----------	------------------------------

Chapter 4 Testing

4.1 Testing

Testing is a critical and systematic phase in the software development lifecycle specifically designed to identify mistakes, defects, and inconsistencies within a system. The primary goal of testing is to ensure that the program will execute its intended tasks effectively, reliably, and consistently under all expected conditions. For FindHome, testing was carried out across all layers of the application - from individual controller functions to the complete integrated system - to verify that every feature performs as designed and that the platform meets both functional and non-functional requirements defined in Chapter 2.

Testing also serves as a quality assurance mechanism that builds confidence in the software before it is made available to end users. Without a structured testing process, defects may go undetected until they cause failures in production, resulting in poor user experience, data loss, or security vulnerabilities. For a platform like FindHome, where sensitive user data, real-time communication, and financial property information are involved, rigorous testing is not optional but essential.

4.2 Overview of Testing

Understanding that testing is a challenging endeavor that requires considerable time and expertise is fundamental to appreciating the value it adds to a software project. While the act of developing a system is considered a commendable task, testing helps us gauge the number

of defects present in the software and provides a structured mechanism for resolving them before deployment.

Testing alone, however, does not guarantee a defect-free system - it helps pinpoint the exact issues present so that developers can address them systematically. Moreover, testing reveals the weaknesses within a system's architecture, security boundaries, and performance characteristics that would otherwise remain hidden during development. For FindHome, the testing phase was treated as an integral part of the development process rather than an afterthought, ensuring that each feature - from user registration and property listing creation to real-time chat and admin controls - was verified against its defined requirements before being considered complete.

A well-structured testing process also ensures that changes made to one part of the system do not inadvertently break other parts. This is particularly important in a full-stack application like FindHome, where the React frontend, Express API, MongoDB database, and Socket.IO real-time server are tightly interconnected and any failure in one tier can cascade across the entire platform.

4.3 Testing Strategy

A strategy for testing FindHome guarantees that the system is dependable, effective, and fulfils all necessary functional and non-functional criteria. The testing strategy adopted for FindHome is comprehensive and multi-layered, covering every aspect of the platform from isolated units of logic to the fully integrated system operating under realistic conditions. The following testing levels were applied:

- i. Testing of Individual Components (Unit Testing)
- ii. Test of Components Working Together (Integration Testing)

- iii. Test of the Entire System (System Testing)
- iv. Test of System Functionality (Validation Testing)
- v. Security Testing
- vi. Performance Testing

Each of these levels targets a different dimension of the system and together they provide complete coverage of all FindHome features and requirements.

4.3.1 Testing of Individual Components (Unit Testing)

Unit testing is the most granular level of testing and forms the foundation of the overall testing strategy. The process of unit testing starts by reviewing every module's interactions to verify their correct functioning in isolation from the rest of the system. For FindHome, each controller function - including those responsible for user registration, login, property listing creation, saved post management, chat initiation, and contact form submission - was tested individually to confirm that it processes valid inputs correctly and returns the expected HTTP status codes and response bodies.

To ensure that the data held in temporary storage remains intact throughout the algorithm's execution, local data structures such as the in-memory OTP store used in the forgot-password flow were scrutinized carefully. Additionally, boundary conditions were verified to confirm that each module successfully controls or limits processing near its operational boundaries. For example, the registration controller was tested with duplicate usernames and duplicate email addresses to confirm that it correctly returns a 409 Conflict response in each case rather than creating duplicate records in MongoDB.

The middleware functions - `verifyToken`, `verifyAdmin`, and `verifyTokenOrAdmin` - were individually verified to accept valid JWT tokens and correctly reject expired, malformed, or absent tokens with appropriate 401 Unauthorized and 403 Forbidden responses. These

middleware units are critical gatekeepers of the FindHome API and their correct functioning underpins the security of the entire platform.

4.3.2 Test of Components Working Together (Integration Testing)

Integration testing examines how individual components of the system function when they are combined, identifying any defects that arise from the interaction between modules that may not be apparent during unit testing. For FindHome, integration tests verified the interaction between the Express routes, controllers, Prisma ORM, and MongoDB working together as a coordinated subsystem.

A full end-to-end API flow was tested: a new user registers through the POST `/api/auth/register` endpoint, logs in through POST `/api/auth/login` to receive a JWT cookie, creates a property listing through POST `/api/posts`, retrieves the listing through GET `/api/posts/:id`, updates its status through PUT `/api/posts/:id`, saves it through POST `/api/saved-posts/:postId`, and finally deletes it through DELETE `/api/posts/:id`. Each step in this chain was verified to produce the correct response and to correctly update the MongoDB database through Prisma.

The cascading delete logic was also verified during integration testing. When a property listing is deleted, the associated `postDetail` document and any `savePost` records referencing that listing must also be removed to prevent orphaned documents in the MongoDB collections. Integration testing confirmed that this cascading deletion operates correctly and completely, maintaining the integrity of the database at all times.

The integration between the Express REST API and the Socket.IO real-time server was also tested, confirming that a message sent through POST `/api/messages/:chatId` is correctly

stored in MongoDB and simultaneously delivered to the recipient through the Socket.IO connection without duplication or loss.

4.3.3 Test of the Entire System (System Testing)

System testing is a form of black-box testing that evaluates the complete operation and efficiency of the FindHome platform as a fully integrated whole. These tests were carried out following the integration phase and prior to the final acceptance phase. The aim is to identify any defects within the system's integrated components and ensure that the entire platform fulfils the specified standards and functions according to user expectations.

For FindHome, system testing was conducted using the full stack simultaneously - the React frontend connected to the Express API, the Prisma-managed MongoDB database, and the Socket.IO real-time server - all running together in a local development environment that mirrors the intended production setup. Key end-to-end flows that were tested include: searching and filtering property listings from the public homepage using city, listing type, property type, price range, and bedroom count filters; navigating to a property detail page and verifying the correct rendering of the Leaflet.js interactive map with accurate GPS coordinates; initiating a real-time chat session from the property detail page and exchanging multiple messages between two browser sessions; managing listings through the owner dashboard including adding a new property, editing its details, changing its status, and deleting it; and completing the full forgot-password flow from submitting the email address through receiving the OTP, verifying it, and successfully resetting the password.

All system-level tests were verified against the original functional requirements defined in Chapter 2 to confirm that every user story has been fully implemented and functions as expected in the complete integrated environment.

4.3.4 Test of System Functionality (Validation Testing)

Validation testing is the method of assessing the FindHome platform against its defined business requirements to confirm that it genuinely fulfils user expectations. Validation testing guarantees that the product truly meets the needs of all stakeholders - property seekers, property owners, and administrators - as identified during the requirement analysis phase in Chapter 2.

For FindHome, validation testing confirmed that the public property search correctly returns only listings with a status of "available" whose owner accounts are active, that the admin dashboard accurately displays platform-wide statistics, that the user dashboard correctly reflects the authenticated user's own listings and saved posts, and that the real-time chat interface correctly tracks unread message counts and displays typing indicators. Each validated feature was verified to perform its designated function when introduced into the correct operating context, confirming that FindHome delivers on the objectives stated in Chapter 1.

4.3.5 Security Testing

Security testing for FindHome was conducted to confirm that all access controls, authentication mechanisms, and data protection measures function correctly and cannot be bypassed by unauthorized users.

Access Permission: It was confirmed that only approved and authenticated users have the ability to create, modify, or delete property listings. Attempts by unauthenticated users to access protected routes correctly returned 401 Unauthorized responses, while attempts by USER-role accounts to access ADMIN-only routes correctly returned 403 Forbidden responses.

Data Protection: It was verified that all sensitive user information - particularly passwords - is stored exclusively in bcrypt-hashed form and never returned in API responses.

The JWT token cookie was confirmed to carry the `HttpOnly` flag, making it inaccessible to JavaScript running in the browser and therefore immune to cross-site scripting attacks. Sensitive environment variables including the JWT secret key, MongoDB connection string, and Resend API key were confirmed to be stored in `.env` files excluded from version control.

Session Management: It was confirmed that sessions are handled correctly through JWT expiry, preventing unauthorized individuals from accessing protected content after a token has expired. The OTP-based forgot-password flow was tested to verify that OTPs expire after ten minutes and cannot be reused once a password has been successfully reset, preventing replay attacks.

Ownership Enforcement: It was confirmed that a user cannot update or delete a property listing belonging to another user. Attempts to do so correctly returned 403 Forbidden responses, ensuring that listing ownership is strictly enforced at the API level regardless of what is presented at the frontend.

4.3.6 Performance Testing

Performance testing was conducted to evaluate how the FindHome platform behaves under realistic operating conditions and to confirm that it meets the performance non-functional requirements defined in Chapter 2.

Load Handling: The system was evaluated for its ability to handle concurrent requests without a significant decline in response times. The `GET /api/posts/active` endpoint, which retrieves all publicly available listings from active owner accounts, was tested with varying filter combinations and consistently returned responses within 300 milliseconds on a local development setup, well within the two-second threshold defined in the non-functional requirements.

Response Time: The duration required to load the property listing page, retrieve a single property detail, and submit a new listing was measured and confirmed to be within acceptable limits. API endpoints involving complex Prisma queries with relational data - such as retrieving a post with its associated postDetail and owner information - returned results in under 500 milliseconds under normal load conditions.

Real-Time Latency: The Socket.IO real-time messaging server was tested for message delivery latency between two connected clients. Messages were delivered with less than 100 milliseconds round-trip latency on a local network, significantly better than the 500-millisecond threshold specified in the non-functional requirements. Typing indicator events were also confirmed to be emitted and received in real time without perceptible delay.

Chapter 5 Tools and Technology

5.1 Introduction to Tools and Technology

In this chapter, we delve into the various tools and technologies employed in the creation of the FindHome real estate platform, focusing on its fundamental components. We explore the key tools, technologies, and approaches employed in the platform's development and deployment, highlighting its robust features, functionality, and superior user interface. Our journey begins with the back-end development phase, during which we utilized Node.js and the Express.js framework, offering a secure and dependable platform for database management and backend logic. Subsequently, we discuss the front-end development, emphasizing the use of React 19 with Vite 7 and Tailwind CSS 4 to achieve a responsive and visually appealing user interface. Furthermore, we explore the role of Socket.IO in improving the user experience through real-time messaging functionality and interactivity. Each of these elements will be discussed in detail below.

The FindHome platform follows a three-tier MERN-like architecture in which the React client, the Express API server, and the MongoDB database operate as independently deployable services. This separation of concerns ensures that each layer can be maintained, scaled, and tested in isolation, contributing to the overall reliability and performance of the platform. The deliberate selection of each tool and technology was guided by the need to build a modern, scalable, and secure real estate platform capable of handling concurrent users, real-time communication, and large volumes of property data.

5.2 Frontend Technologies

The frontend of FindHome is responsible for delivering a fast, responsive, and visually consistent experience to all users regardless of their device or browser. The following technologies were selected to achieve this goal.

5.2.1 React 19

React is a declarative, component-based JavaScript library maintained by Meta that enables the construction of dynamic and interactive user interfaces. FindHome uses React 19, the latest stable release, to build all client-side views including the home page, listing detail pages, dashboard, authentication screens, and real-time chat interface. React's virtual DOM ensures that only the components affected by a state change are re-rendered, resulting in a highly performant user experience. The component-based architecture also promotes reusability and maintainability across the entire frontend codebase.

The benefits of using React 19 in FindHome include:

- i. Declarative and component-driven UI development
- ii. Virtual DOM for optimized rendering performance
- iii. Large ecosystem with extensive third-party library support
- iv. Seamless integration with Vite for fast development builds

5.2.2 Vite 7

Vite is a next-generation frontend build tool that provides an extremely fast development server and optimized production builds. FindHome uses Vite 7 as its build tool and development server, replacing the slower traditional bundlers. Vite leverages native ES modules in the browser during development, enabling near-instantaneous hot module replacement (HMR). This significantly reduces the feedback loop during development and accelerates the overall development process.

The benefits of using Vite 7 in FindHome include:

- i. Near-instant server startup and hot module replacement
- ii. Optimized production builds using Rollup under the hood

- iii. Native ES module support for modern browsers
- iv. Minimal configuration required for React projects

5.2.3 Tailwind CSS 4

Tailwind CSS is a utility-first CSS framework that enables developers to build custom designs directly in their markup without writing separate stylesheets. FindHome uses Tailwind CSS 4 to style all components, ensuring a consistent visual language across the entire platform. Tailwind's responsive utility classes make it straightforward to adapt layouts for desktop, tablet, and mobile viewports without the overhead of a traditional CSS framework like Bootstrap.

The benefits of using Tailwind CSS 4 in FindHome include:

- i. Utility-first approach eliminates the need for custom CSS files
- ii. Built-in responsive design utilities for all screen sizes
- iii. Smaller production CSS bundles through automatic unused style purging
- iv. Highly consistent design system enforced through configuration

5.2.4 React Router DOM 7

React Router DOM is the standard client-side routing library for React applications. FindHome uses React Router DOM 7 to manage all navigation between pages including the home page, listing detail pages, authentication screens, the admin dashboard, and the user profile page. It enables single-page application behaviour where navigation between views does not require a full page reload, delivering a faster and smoother user experience.

The benefits of using React Router DOM 7 in FindHome include:

- i. Declarative route definitions with nested route support
- ii. Programmatic navigation using the `useNavigate` hook
- iii. Protected route implementation for authentication-gated pages
- iv. URL parameter extraction for dynamic listing detail pages

5.2.5 Axios

Axios is a promise-based HTTP client for JavaScript that simplifies communication between the frontend and backend API. FindHome uses Axios to handle all API requests including user authentication, property listing retrieval, profile updates, and administrative operations. Axios is configured with a base URL and the `withCredentials` flag to automatically include the HTTP-only JWT cookie in every request, ensuring seamless authentication without manual token management.

The benefits of using Axios in FindHome include:

- i. Promise-based API with `async/await` support
- ii. Automatic JSON serialization and deserialization
- iii. Interceptor support for centralized error handling
- iv. Credential forwarding for cookie-based authentication

5.2.6 Leaflet.js

Leaflet.js is a lightweight, open-source JavaScript library for interactive maps. FindHome integrates Leaflet.js on property detail pages to render an interactive map pinpointing the exact location of each listing using the GPS latitude and longitude coordinates stored in the database. This provides buyers and renters with a clear visual understanding of a property's location relative to nearby landmarks, schools, bus stops, and restaurants.

The benefits of using Leaflet.js in FindHome include:

- i. Lightweight library with no heavy dependencies
- ii. Interactive map with zoom, pan, and marker support
- iii. Open-source with free tile layer support via OpenStreetMap
- iv. Simple React integration through the `react-leaflet` wrapper

5.2.7 Socket.IO Client

Socket.IO is a library that enables real-time, bidirectional, event-based communication between the browser and the server. FindHome integrates the Socket.IO client on the frontend

to power the real-time chat system, allowing buyers and property owners to exchange messages instantly without page refreshes. The client connects to the dedicated Socket.IO server and listens for incoming message events, typing indicator broadcasts, and online user status updates.

The benefits of using Socket.IO Client in FindHome include:

- i. Real-time bidirectional communication over WebSockets
- ii. Automatic fallback to HTTP long-polling when WebSockets are unavailable
- iii. Event-driven architecture for clean separation of chat logic
- iv. Built-in support for typing indicators and online presence tracking

5.3 Backend Technologies

The backend of FindHome is responsible for processing all business logic, handling API requests, managing authentication, and facilitating real-time communication. The following technologies were selected to build a secure, scalable, and maintainable server-side architecture.

5.3.1 Node.js

Node.js is a cross-platform, open-source JavaScript runtime environment built on Chrome's V8 engine that allows developers to execute JavaScript on the server side. FindHome uses Node.js as the runtime for both the Express API server and the Socket.IO real-time server. Node.js's non-blocking, event-driven I/O model makes it particularly well-suited for handling concurrent API requests and maintaining persistent WebSocket connections simultaneously without performance degradation.

The benefits of using Node.js in FindHome include:

- i. Non-blocking I/O for high concurrency handling
- ii. Unified JavaScript language across frontend and backend
- iii. Large ecosystem of packages available through npm
- iv. Excellent performance for real-time and API-driven applications

5.3.2 Express.js

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for building web APIs. FindHome uses Express.js to define and handle all REST API routes, including authentication, property listing management, user administration, chat operations, and contact form submissions. Express middleware is used throughout the application for JSON parsing, cookie parsing, CORS configuration, and JWT-based route protection.

The benefits of using Express.js in FindHome include:

- i. Minimal and unopinionated framework allowing full architectural control
- ii. Powerful middleware system for request preprocessing and authentication
- iii. Straightforward REST API route definition with HTTP method support
- iv. Wide compatibility with Node.js ecosystem packages

5.3.3 Prisma ORM 6

Prisma is a next-generation, type-safe Object Relational Mapper (ORM) that simplifies database access in Node.js and TypeScript applications. FindHome uses Prisma ORM 6 with its MongoDB connector to define the entire database schema in a single `schema.prisma` file and generate a fully type-safe client for all database queries. Prisma replaces raw MongoDB queries with a clean and readable API, reducing the risk of query errors and improving developer productivity.

The benefits of using Prisma ORM 6 in FindHome include:

- i. Type-safe database queries eliminating runtime query errors
- ii. Single schema file as the single source of truth for the data model
- iii. Auto-generated Prisma Client with IntelliSense support
- iv. Native MongoDB support with ObjectId and embedded document handling

5.3.4 JSON Web Tokens (JWT)

JSON Web Tokens are an open standard for securely transmitting information between parties as a digitally signed JSON object. FindHome uses JWT for stateless user authentication. Upon successful login, the server generates a signed JWT containing the user's ID and role, which is stored as an HTTP-only cookie on the client. All protected API routes validate this token through a `verifyToken` middleware before processing the request, ensuring that only authenticated users can access sensitive operations.

The benefits of using JWT in FindHome include:

- i. Stateless authentication requiring no server-side session storage
- ii. HTTP-only cookie storage preventing JavaScript-based token theft
- iii. Role-based claims enabling admin and user access control
- iv. Compact and self-contained token format for efficient transmission

5.3.5 bcrypt

bcrypt is a password hashing library based on the Blowfish cipher that is specifically designed for securely hashing and salting user passwords. FindHome uses bcrypt with a salt round value of ten to hash all user passwords before storing them in the database. During login, the submitted password is compared against the stored hash using bcrypt's `compare` function, ensuring that plaintext passwords are never stored or transmitted.

The benefits of using bcrypt in FindHome include:

- i. Adaptive hashing algorithm that increases in cost over time
- ii. Built-in salt generation preventing rainbow table attacks
- iii. Industry-standard approach to secure password storage
- iv. Simple API for hashing and comparison operations

5.3.6 Resend

Resend is a modern developer-focused email delivery API that enables applications to send transactional emails reliably. FindHome uses Resend to deliver OTP emails during the

forgot-password flow. When a user requests a password reset, the server generates a six-digit OTP, hashes it using bcrypt, stores the hash and expiry in the database, and then delivers the plaintext OTP to the user's registered email address via the Resend API.

The benefits of using Resend in FindHome include:

- i. Simple REST-based email delivery API with minimal configuration
- ii. High deliverability rates for transactional emails
- iii. Developer-friendly SDK with async/await support
- iv. Reliable email delivery essential for the OTP-based password reset flow

5.3.7 Socket.IO Server

Socket.IO Server is the server-side counterpart of the Socket.IO library that manages WebSocket connections and event broadcasting. FindHome runs a dedicated Socket.IO server alongside the Express API to handle all real-time chat functionality. The server maintains a map of online users, broadcasts typing indicators to chat participants, delivers new messages in real time, and emits online user status updates to connected clients.

The benefits of using Socket.IO Server in FindHome include:

- i. Real-time event broadcasting to specific users or rooms
- ii. Persistent connection management for online user tracking
- iii. Typing indicator events for an engaging chat experience
- iv. Fallback transport support ensuring connectivity across network conditions

5.4 Database

MongoDB is a leading open-source, document-oriented NoSQL database that stores data in flexible, JSON-like BSON documents rather than traditional rows and columns. FindHome uses MongoDB as its primary database to store all user profiles, property listings, listing details, saved posts, chat conversations, messages, and contact form submissions. MongoDB's schema-

less nature and support for nested documents and arrays make it well-suited for the variable structure of real estate listing data, where different property types may carry different sets of attributes.

MongoDB is accessed in FindHome exclusively through the Prisma ORM, which enforces a consistent schema definition and provides type-safe query operations. The database is hosted on MongoDB Atlas, Mongo's fully managed cloud database service, which provides automatic scaling, daily backups, and a globally distributed cluster infrastructure. This setup ensures that FindHome's data layer is both reliable and capable of growing with an increasing user base.

The benefits of using MongoDB in FindHome include:

- i. Flexible document model accommodating varied property data structures
- ii. Native support for arrays enabling multi-image storage per listing
- iii. High scalability through horizontal sharding on MongoDB Atlas
- iv. Seamless integration with Prisma ORM for type-safe queries

5.5 Development Tools

5.5.1 Visual Studio Code (VS Code)

Microsoft developed Visual Studio Code, an open-source, free code editor designed for coding, debugging, and managing source control. FindHome was developed entirely using VS Code as the primary code editor for both the React frontend and the Node.js backend. VS Code's rich extension ecosystem, including Prisma language support, ESLint, and Prettier, significantly enhanced developer productivity and code quality throughout the project.

The benefits of using VS Code in FindHome include:

- i. Lightweight yet feature-rich development environment
- ii. Built-in Git integration for version control management
- iii. IntelliSense code completion for JavaScript, React, and Prisma
- iv. Works across Windows, macOS, and Linux platforms

5.5.2 Postman

Postman is a widely used API development and testing platform that allows developers to create, send, and inspect HTTP requests. FindHome's entire REST API was tested and documented using Postman during the backend development phase. All authentication endpoints, listing CRUD operations, admin routes, and chat API endpoints were verified through Postman before frontend integration, ensuring that the API behaved correctly and returned the expected responses.

The benefits of using Postman in FindHome include:

- i. Intuitive interface for constructing and sending HTTP requests
- ii. Environment variable support for switching between development and production URLs
- iii. Collection organization for grouping related API endpoints
- iv. Detailed response inspection including headers, cookies, and body

5.5.3 Git and GitHub

Git is a distributed version control system that tracks changes in source code during software development. FindHome used Git for all source code version control throughout the development lifecycle. GitHub served as the remote repository hosting platform, enabling safe storage of the codebase and providing a history of all changes made to both the frontend and backend codebases.

The benefits of using Git and GitHub in FindHome include:

- i. Complete history of all code changes with rollback capability
- ii. Branch-based development enabling parallel feature implementation
- iii. Remote repository hosting ensuring code backup and accessibility
- iv. Industry-standard tool familiar to all collaborating developers

5.5.4 MongoDB Atlas

MongoDB Atlas is the fully managed cloud database service provided by MongoDB, Inc. FindHome uses MongoDB Atlas to host the production MongoDB database, eliminating the need to manage database infrastructure manually. Atlas provides a web-based dashboard for monitoring database performance, managing collections, running queries, and configuring network access rules. The free tier of MongoDB Atlas was sufficient to host FindHome's database during development and testing phases.

The benefits of using MongoDB Atlas in FindHome include:

- i. Fully managed database hosting with automatic backups
- ii. Web-based collection browser and query interface
- iii. Built-in performance monitoring and alerting tools
- iv. Scalable cluster tiers supporting growth from development to production

Chapter 6 User Interface Design

6.1 Introduction to User Interface Design

The user interface (UI) plays a crucial role in every web platform as it directly affects the experience and interaction of users. This section provides an in-depth overview of the design concepts, structure, and operation of the FindHome real estate platform. The aim is to create a coherent and attractive user experience that simplifies the process of discovering properties, communicating with owners, and managing listings for buyers, renters, property owners, and administrators. FindHome's UI was built using React 19 and Tailwind CSS 4, ensuring a fully responsive and visually consistent interface across desktop, tablet, and mobile devices.

The platform is structured around two primary layouts. The 'AppLayout' wraps all public-facing pages with a shared header and footer, while the 'UserDashboardLayout' wraps all authenticated dashboard pages with a sidebar navigation panel. Client-side routing is managed entirely by React Router DOM 7, enabling seamless single-page application navigation without full page reloads.

6.2 Public Pages

The following pages are nested under the AppLayout and are accessible to all visitors without requiring authentication. They are served under the root path and include the header and footer on every page.

6.2.1 Home Page

The Home Page is the primary landing page of FindHome, served at the root path /. It presents a hero section with a prominent search bar that allows visitors to filter properties by city, listing type (Buy or Rent), property type, minimum price, and maximum price. Below the hero section, a curated grid of recently added property listings is displayed, each card showing the property image, title, price, address, number of bedrooms, and bathrooms. The home page serves as the central discovery hub for all prospective buyers and renters visiting the platform for the first time.

6.2.2 About Page

The About Page, served at /about, provides visitors with an overview of the FindHome platform, its mission, and the values that drive its development. It introduces the purpose of the platform, explains how FindHome bridges the gap between property owners and buyers or renters, and highlights the key advantages of using FindHome over traditional fragmented real estate listing services. This page helps build trust and credibility with new visitors exploring the platform.

6.2.3 List Page

The List Page, served at /list, displays all available property listings in a dual-panel layout. The left panel shows a scrollable list of property cards reflecting all listings currently in the database. The right panel renders an interactive Leaflet.js map with markers pinpointing the exact GPS coordinates of each listed property. This page provides buyers and renters with both a list-based and map-based view of all available properties simultaneously.

6.2.4 Search Results Page

The Search Results Page, served at /search-results, displays property listings filtered according to the search criteria submitted by the user from the Home Page search bar. Results are filtered by city, listing type, property type, bedroom count, and price range. The page renders a responsive grid of matching listing cards, each showing key property information. If no listings match the submitted criteria, an appropriate empty state message is displayed to guide the user back to the search form.

6.2.5 Property Detail Page

The Property Detail Page, served at `/property-detail/:id`, provides a comprehensive view of a single property listing loaded dynamically using the `singlePropertyLoader` React Router loader function. It displays a full image gallery, the property title, price, address, listing type, property type, number of bedrooms and bathrooms, and property size. Below the primary details, an extended information section presents the full property description, utilities policy, pet policy, income requirement, and proximity data including distances to the nearest school, bus stop, and restaurant in metres. An interactive Leaflet.js map is embedded on this page, pinpointing the exact GPS location of the property. Authenticated users can initiate a real-time chat with the property owner directly from this page.

6.2.6 Login Page

The Login Page, served at `/login`, presents a clean authentication form accepting the user's registered email address and password. Upon successful authentication, the server issues a signed JWT stored as an HTTP-only cookie, and the user is redirected to their dashboard overview. The page also provides a navigational link to the Forgot Password page for users who have lost access to their accounts. This page ensures that only authorized users can access protected features such as listing management, saved posts, and real-time chat.

6.2.7 Register Page

The Register Page, served at `/register`, allows new visitors to create a FindHome account by supplying their chosen username, email address, and password. Upon successful registration, the user's password is hashed using `bcrypt` before being stored in the database, and the user is redirected to the login page. The registration form includes client-side validation to ensure all required fields are completed correctly before submission.

6.2.8 Forgot Password Page

The Forgot Password Page, served at `/forgotpassword`, implements a secure three-step account recovery flow. In the first step, the user enters their registered email address to request a password reset. The server generates a six-digit OTP, hashes it using `bcrypt`, stores the hash alongside an expiry timestamp in the database, and delivers the plaintext OTP to the user's email address via the Resend API. In the second step, the user enters the received OTP for server-side verification against the stored hash. In the third step, the user sets and confirms a

new password, which is then hashed and persisted to the database. This three-step flow ensures secure account recovery without exposing sensitive credentials at any stage.

6.2.9 Contact Page

The Contact Page, served at /contact, provides a publicly accessible enquiry form through which any visitor can submit a message to the FindHome administration team. The form collects the visitor's full name, email address, optional phone number, subject line, and message body. Upon submission, the enquiry is stored in the ContactMessage collection in the database with an isRead flag defaulting to false. Administrators can view, manage, and mark as read all submitted enquiries through the Queries page in the admin dashboard.

6.2.10 Privacy Policy Page

The Privacy Policy Page, served at /privacy-policy, presents the platform's data handling and privacy practices to all users and visitors. It outlines how FindHome collects, uses, stores, and protects user data in compliance with applicable privacy standards. This page is accessible from the footer of every public page and is an essential component of the platform's legal transparency framework.

6.2.11 Terms of Service Page

The Terms of Service Page, served at /terms-of-service, outlines the rules, obligations, and conditions that govern the use of the FindHome platform. It defines the responsibilities of users, property owners, and administrators, and specifies the acceptable use policies of the platform. Like the Privacy Policy page, it is accessible from the footer of every public page and forms part of the platform's legal documentation.

6.2.12 Sitemap Page

The Sitemap Page, served at /sitemap, provides a structured overview of all publicly accessible pages and routes within the FindHome platform. It assists visitors in quickly locating specific sections of the platform and supports search engine indexing by making the full page hierarchy visible. The sitemap lists all major public routes including the home page, property listings, about, contact, authentication pages, and legal pages.

6.3 Dashboard Pages

All dashboard pages are nested under the `UserDashboardLayout` at the `/dashboard` path. The layout wraps every dashboard page with a persistent sidebar navigation panel. Access to all dashboard routes is enforced on the backend through two middleware functions defined in `verifyToken.js`. The `verifyToken` middleware validates the JWT stored as an HTTP-only cookie and attaches the user's ID and admin status to every request. The `verifyAdmin` middleware extends this by additionally checking whether `req.isAdmin` is true and returning a 403 Forbidden response if not. Navigating to `/dashboard` directly automatically redirects the user to `/dashboard/overview`.

The dashboard pages are divided into three access tiers:

- **Both Users and Admins** - pages accessible to any authenticated user regardless of role
- **Owner only (self) or Admin** - pages where a regular user can only access their own data, but an admin can access anyone's
- **Admin only** - pages restricted exclusively to users with the ADMIN role

6.3.1 Overview Page (*Both Users and Admins*)

The Overview Page, served at `/dashboard/overview`, is the first page every authenticated user lands on after login. For regular users, it displays a personal activity summary including the count of their active listings, the number of properties they have saved, and any unread chat messages. For administrators, the same page additionally displays platform-wide statistics drawn from the `getPostStats` controller, which accepts an `isAdmin` query parameter. When `isAdmin` is true, the controller removes the user filter entirely and returns counts across all listings on the platform grouped by status: available, sold, pending, and rented. This dual behaviour makes the Overview page the operational hub for both regular users and platform administrators.

6.3.2 My Properties Page (*Owner only or Admin*)

The My Properties Page, served at `/dashboard/myProperties`, displays all property listings belonging to the currently authenticated user. On the backend, the `getUserPosts` controller enforces an ownership check: if the requesting token's user ID does not match the target user ID and the requester is not an admin, a 403 Forbidden response is returned. Administrators can therefore view the listings of any user, while regular users can only view

their own. Each listing entry on this page displays the property title, price, city, listing type, current status, and creation date alongside action buttons for navigating to the Edit Property page or permanently deleting the listing.

6.3.3 Add Property Page (Both Users and Admins)

The Add Property Page, served at `/dashboard/addProperty`, provides any authenticated user with a comprehensive two-section form for creating a new property listing. The first section collects core listing fields: title, price, address, city, GPS latitude and longitude, number of bedrooms and bathrooms, property type, listing type, status, and image URLs. The second section collects extended `postDetail` fields: full description, utilities policy, pet policy, minimum income requirement, property size in square metres, and proximity distances to the nearest school, bus stop, and restaurant. On the backend, the `createPost` controller associates the new listing with the requesting user's ID extracted from the verified JWT, ensuring every listing has a clearly identified owner. Upon successful submission, both a post and a nested `postDetail` record are created in the database simultaneously.

6.3.4 Edit Property Page (*Owner only or Admin*)

The Edit Property Page, served at `/dashboard/edit/:postId`, allows a property owner to modify an existing listing identified by its dynamic `postId` URL parameter. The form is pre-populated with all current values from the selected listing, covering both core post fields and extended `postDetail` fields. On the backend, the `updatePost` controller uses the `canModify` helper function which returns true only if `isAdmin` is true or the post's `userId` matches the requesting token's user ID. This means a regular user who did not create the listing will receive a 403 Forbidden response if they attempt to edit it, while an administrator can edit any listing on the platform. The controller uses Prisma's `upsert` operation on the `postDetail` relation, creating the detail record if it does not exist or updating it if it does. The listing status can also be changed from this page, allowing owners to mark properties as available, pending, sold, or rented.

6.3.5 Saved Posts Page (*Both Users and Admins*)

The Saved Posts Page, served at `/dashboard/favorites`, displays all property listings that the currently authenticated user has bookmarked. The backend `savePost` controller enforces that saved posts are always scoped to the requesting user's own ID extracted from the JWT,

meaning each user can only see and manage their own saved collection. Each saved listing card displays the property image, title, price, address, and a button to remove the listing from the saved collection. This page allows buyers and renters to maintain a personalized shortlist of properties they are actively considering, making it easy to compare and revisit listings of interest without performing repeated searches.

6.3.6 Messages Page (Both Users and Admins)

The Messages Page, served at `/dashboard/messages`, provides any authenticated user with a full real-time chat interface powered by Socket.IO. The page is initialized using the `profilePageLoader` React Router loader, which pre-fetches the user's profile and existing chat list before the component renders, preventing a blank loading flash. The left panel displays all active conversations, showing each participant's username, avatar, the last message preview, and an unread message count badge derived from the `seenBy` array on each chat record. The right panel renders the full message history of the selected conversation, with messages grouped by sender and displayed with timestamps. A text input field at the bottom allows users to compose and send new messages. The Socket.IO server broadcasts typing indicator events to the other participant in real time when a user is composing a reply, delivering a fully interactive and responsive messaging experience. Both regular users and administrators use the same Messages page since chat is a general platform feature available to all authenticated users.

6.3.7 Profile Page (Both Users and Admins)

The Profile Page, served at `/dashboard/profile`, allows any authenticated user to view and update their own account information. On the backend, the `updateUser` controller enforces a self-ownership check: it returns 403 Forbidden if the requesting token's user ID does not match the target user ID and the requester is not an admin. This means regular users can only update their own profile, while administrators can update any user's profile including toggling the `isActive` flag and reassigning the `role` field, which are fields the controller only permits to be updated when `req.isAdmin` is true. The page displays the user's current username, email address, phone number, and profile avatar alongside an editable form pre-filled with existing values. Users can also permanently delete their own account from this page, which triggers the `deleteUser` controller's cascading deletion of all associated posts, `postDetails`, `savedPosts`, chat memberships, and messages before removing the user record and clearing the JWT cookie.

6.3.8 Users Page (Admin only)

The Users Page, served at `/dashboard/users`, is protected on the backend by the `verifyAdmin` middleware and is therefore accessible exclusively to users with the ADMIN role. Any request to the underlying API endpoint from a non-admin user returns a 403 Forbidden response. The page displays a complete list of all registered users on the platform, retrieved through the `getAllUsers` controller which fetches every user record and includes a count of their associated posts using Prisma's `_count` aggregation. Each user entry displays the username, email address, assigned role, registration date, post count, and current `isActive` status. Administrators can toggle any user's `isActive` status through the `updateUser` controller, which immediately controls whether that user's listings appear publicly on the platform. Administrators can also permanently delete any user account from this page, triggering the full cascading deletion sequence in the `deleteUser` controller.

6.3.9 Property Detail Page - Dashboard View (Both Users and Admins)

The Property Detail Page within the dashboard, served at `/dashboard/property/:id`, allows authenticated users to view the complete details of any single property listing without leaving the dashboard layout. It is populated by the `getPost` controller which fetches the full listing record including the associated `postDetails` and owner information. Since `getPost` does not apply any ownership restriction, both regular users and administrators can view the full detail of any listing through this route. This page gives administrators a convenient way to review a complete listing in full context before deciding to update its status or remove it from the platform, and gives regular users a clean in-dashboard view of any listing they wish to inspect.

6.3.10 Queries Page (Admin only)

The Queries Page, served at `/dashboard/queries`, is protected on the backend by the `verifyAdmin` middleware and is accessible exclusively to administrators. It displays all enquiries submitted through the public Contact Page, managed by the `contact.controller.js`. Each submission entry shows the sender's full name, email address, phone number, subject, message body, and submission timestamp. Administrators can mark individual submissions as read, which updates the `isRead` flag in the `ContactMessage` collection and removes the

unread indicator from the Overview dashboard summary. This page ensures that all incoming user enquiries are systematically tracked, reviewed, and addressed by the platform administration team in a timely manner.

Chapter 7 Conclusion and Future Work

7.1 Conclusion of a Student Management System (SMS) Implementation

The introduction of FindHome - a full-stack real estate platform - greatly enhances the handling and organization of property listings and user interactions in the modern property market. By automating essential tasks such as user registration and authentication, property listing management, real-time communication between buyers and property owners, saved post tracking, and administrative oversight, the platform reduces the possibility of errors made through manual processes, makes property data easier to access, and offers detailed statistics on listing performance, aiding owners and administrators in making more informed decisions. Additionally, the system supports role-based access control through JWT authentication and middleware-enforced route protection, guaranteeing the safety, integrity, and accuracy of all user and property data stored in the MongoDB database.

The platform's three-tier architecture - consisting of a React 19 frontend, an Express.js API server, and a MongoDB database managed through Prisma ORM - ensures that each layer of the system operates independently and can be scaled or maintained without disrupting the others. The integration of Socket.IO delivers a seamless real-time messaging experience, while the Resend-powered OTP email flow provides a secure and user-friendly account recovery mechanism. The interactive Leaflet.js maps and proximity data features further differentiate FindHome from traditional static listing platforms by giving buyers and renters meaningful spatial context for every property they explore.

In summary, FindHome creates a more effective and well-organized environment for the management of property listings, buyer-owner communication, and platform administration, easing the operational burden on property owners and administrators while significantly enhancing the overall experience for every user involved in the property discovery and transaction process.

7.2 Future Work

Even though the platform is performing well in its current state, there are many opportunities to improve and expand FindHome going forward:

7.2.1 Integration with Payment Gateways

Linking FindHome with online payment platforms such as Stripe or PayPal to enable secure in-platform rent payments, booking deposits, and transaction history tracking. This would allow buyers and renters to complete financial commitments directly through the platform without relying on external payment arrangements, creating a fully end-to-end property transaction experience.

7.2.2 Development of a Mobile Application

The creation of a dedicated mobile application for buyers, renters, and property owners to access FindHome anytime and from any location, with push notifications for critical updates such as new messages, listing status changes, saved post availability updates, and contact form responses. A mobile app would significantly broaden the platform's reach and accessibility for users who primarily browse property listings on smartphones.

7.2.3 Artificial Intelligence-Driven Property Recommendations

The implementation of AI and machine learning technologies to analyse user browsing behaviour, saved posts, and search history for predictive recommendation analysis. This would enable FindHome to proactively suggest properties that match each user's demonstrated preferences, identifying listings they are most likely to be interested in and presenting them at the top of search results or on the dashboard overview page.

7.2.4 Customizable Dashboards

The addition of further customizable dashboard views for different user roles, providing tailored data visualizations and quick-access panels for the most frequently used features. For

property owners, this could include earnings projections and listing performance graphs. For administrators, it could include user growth charts, platform-wide listing trend analysis, and contact query response time metrics.

7.2.5 Cloud-Based Infrastructure and Deployment

Migrating FindHome's API and Socket.IO servers to a fully managed cloud infrastructure such as AWS, Google Cloud, or Microsoft Azure for greater flexibility, automatic scaling, and higher availability. This would enable the platform to handle significantly larger volumes of concurrent users, property listings, and real-time chat connections without performance degradation, accommodating growth from a regional platform to a nationally distributed service.

7.2.6 Support for Multiple Languages

The inclusion of multi-language support to accommodate buyers, renters, and property owners from diverse linguistic backgrounds. Implementing internalization through a library such as `i18next` would allow the FindHome interface to be rendered in multiple languages, making the platform accessible to a broader and more diverse user base across different regions and communities.

7.2.7 Advanced Search and Filtering

The expansion of the current search and filtering system to include additional criteria such as property age, furnished or unfurnished status, proximity radius filtering using GPS coordinates, virtual tour availability, and neighbourhood safety ratings. Advanced filtering would give buyers and renters significantly more control over their property discovery experience and reduce the time required to find listings that precisely match their requirements.

7.2.8 Extended Platform Scope

The current FindHome platform is designed to serve individual property owners and buyers within a defined market scope. In the future, the platform can be extended to support real estate agencies and property management companies, enabling multi-agent listing management, agency-branded profiles, bulk listing uploads, and agency-level administrative dashboards. This extension would position FindHome as a comprehensive solution for both individual and institutional participants in the real estate market.

By concentrating on these future developments, FindHome can continue to grow and adapt to the evolving demands of the contemporary real estate market, delivering increasing value to buyers, renters, property owners, and platform administrators alike.

Chapter 8 References

Axios. (2024). *Axios - Promise Based HTTP Client for the Browser and Node.js*. Retrieved from npm Registry: <https://www.npmjs.com/package/axios>

bcrypt. (2024). *bcrypt - A Library to Help You Hash Passwords in Node.js*. Retrieved from npm Registry: <https://www.npmjs.com/package/bcrypt>

cookie-parser. (2024). *cookie-parser - Parse HTTP Request Cookies in Express*. Retrieved from npm Registry: <https://www.npmjs.com/package/cookie-parser>

cors. (2024). *cors - Node.js CORS Middleware for Express*. Retrieved from npm Registry: <https://www.npmjs.com/package/cors>

dotenv. (2024). *dotenv - Loads Environment Variables from a .env File into process.env*. Retrieved from npm Registry: <https://www.npmjs.com/package/dotenv>

Express.js. (2024). *Express - Fast, Unopinionated, Minimalist Web Framework for Node.js*. Retrieved from Express.js Official Documentation: <https://expressjs.com/>

jsonwebtoken. (2024). *jsonwebtoken - An Implementation of JSON Web Tokens for Node.js*. Retrieved from npm Registry: <https://www.npmjs.com/package/jsonwebtoken>

Leaflet.js. (2024). *Leaflet - An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps*. Retrieved from Leaflet.js Official Documentation: <https://leafletjs.com/>

Meta Open Source. (2024). *React - A JavaScript Library for Building User Interfaces*. Retrieved from React Official Documentation: <https://react.dev/>

MongoDB. (2024). *MongoDB - What is MongoDB? Document Database Overview*. Retrieved from MongoDB Official Documentation: <https://www.mongodb.com/docs/manual/introduction/>

nodemailer. (2024). *Nodemailer - Send Emails from Node.js Applications*. Retrieved from Nodemailer Official Documentation: <https://nodemailer.com/about/>

Node.js Foundation. (2024). *Node.js - About Node.js Asynchronous Event-Driven JavaScript Runtime*. Retrieved from Node.js Official Documentation: <https://nodejs.org/en/about/>

Prisma. (2024). *Prisma ORM - Next-Generation Node.js and TypeScript ORM*. Retrieved from Prisma Official Documentation: <https://www.prisma.io/docs/>

React Router. (2024). *React Router DOM - Declarative Routing for React Applications*. Retrieved from React Router Official Documentation: <https://reactrouter.com/en/main>

Resend. (2024). *Resend - The Email API Built for Developers*. Retrieved from Resend Official Documentation: <https://resend.com/docs/introduction>

Socket.IO. (2024). *Socket.IO - Bidirectional and Low-Latency Communication for Every Platform*. Retrieved from Socket.IO Official Documentation: <https://socket.io/docs/v4/>

Tailwind CSS. (2024). *Tailwind CSS - A Utility-First CSS Framework for Rapid UI Development*. Retrieved from Tailwind CSS Official Documentation: <https://tailwindcss.com/docs/installation>

Vite. (2024). *Vite - Next Generation Frontend Tooling for Modern Web Projects*. Retrieved from Vite Official Documentation: <https://vitejs.dev/guide/>